EXPERIMENT-66(HDOC)

C Programming

Program: GCD of Two Positive Integers Using Euclid's Algorithm and Recursion

Step 1: Understand the Problem Statement

- Input: Two positive integers a and b. A zero, a negative number or a non-numeric value is rejected with an error message.
- Output: The greatest common divisor (GCD) of the two numbers, displayed as `GCD of a and b = result`.
- The GCD of two numbers is the largest number that divides both of them without a remainder. It is found with the recursive function `gcd(int a, int b)`.
- Euclid's algorithm is based on the fact that the GCD does not change when the larger number is replaced by its remainder on division by the smaller number, so `gcd(a, b) = gcd(b, a % b)` and the process repeats until the remainder becomes 0.
- Base case: when b is 0, the answer is a. Recursive case: otherwise `gcd()` calls itself with the pair `(b, a % b)`. The second number strictly decreases at every call, so the recursion always ends. If a is smaller than b, the first call simply swaps them (a % b = a).

Rule(s) used: gcd(a, b) = a when b = 0; gcd(a, b) = gcd(b, a mod b) when b > 0. Example: gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) = 6.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`First number: 48` `Second number: 18`	GCD of 48 and 18 = 6	GCD of 48 and 18 = 6	Pass
2	`First number: 17` `Second number: 13`	GCD of 17 and 13 = 1	GCD of 17 and 13 = 1	Pass
3	`First number: 100` `Second number: 25`	GCD of 100 and 25 = 25	GCD of 100 and 25 = 25	Pass
4	`First number: 7` `Second number: 7`	GCD of 7 and 7 = 7	GCD of 7 and 7 = 7	Pass
5	`First number: 36` `Second number: 60`	GCD of 36 and 60 = 12	GCD of 36 and 60 = 12	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
6	`First number: 0`	Invalid input. Enter positive integers only.	Invalid input. Enter positive integers only.	Pass
7	`First number: 10` `Second number: -4`	Invalid input. Enter positive integers only.	Invalid input. Enter positive integers only.	Pass

Step 3: Algorithm

1. Start the program.
2. Read the first number `a`; if it is not a positive integer, display an error message and stop.
3. Read the second number `b`; if it is not a positive integer, display an error message and stop.
4. Call `gcd(a, b)`.
5. In `gcd()`: if `b == 0`, return a (base case).
6. Otherwise return `gcd(b, a % b)` (recursive case): the pair is replaced by the second number and the remainder of the division.
7. When the recursion returns, display `GCD of a and b = result`.
8. Stop the program.

Evaluation against Step 2 test cases: for (48, 18) the calls are gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0), which returns 6. For (17, 13) the calls are gcd(17, 13) → gcd(13, 4) → gcd(4, 1) → gcd(1, 0) = 1, so the numbers are co-prime. For (100, 25) the remainder is 0 at the first step, giving gcd(25, 0) = 25, and for (7, 7) it gives gcd(7, 0) = 7. For (36, 60) the first call becomes gcd(60, 36), then gcd(36, 24) → gcd(24, 12) → gcd(12, 0) = 12. A zero as the first number and a negative second number are rejected as invalid input — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-67: GCD of two positive integers using Euclid's algorithm
   and RECURSION.

   Euclid's algorithm:  gcd(a, b) = gcd(b, a % b)   and   gcd(a, 0) = a
   (the GCD of two numbers does not change when the larger number is
   replaced by its remainder on division by the smaller one).

   Base case     : b == 0        -> the answer is a
   Recursive case: b != 0        -> repeat with the pair (b, a % b)
   The second number strictly decreases at every call, so the recursion
   always reaches the base case.
*/

#include <stdio.h>

/* Function prototype */
```

```c
int gcd(int a, int b);

int main() {
    int a, b;

    printf("Enter first number: ");
    if (scanf("%d", &a) != 1 || a <= 0) {
        printf("Invalid input. Enter positive integers only.\n");
        return 1;
    }

    printf("Enter second number: ");
    if (scanf("%d", &b) != 1 || b <= 0) {
        printf("Invalid input. Enter positive integers only.\n");
        return 1;
    }

    printf("GCD of %d and %d = %d\n", a, b, gcd(a, b));

    return 0;
}

/* Returns the GCD of a and b using Euclid's algorithm recursively */
int gcd(int a, int b) {
    if (b == 0)                     /* base case: gcd(a, 0) = a */
        return a;
    return gcd(b, a % b);           /* recursive case: gcd(b, a mod b) */
}
```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`gcc hdoc67.c`	(compile)
`./a.out`	(execute)

All 7 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```
 Session   Actions    Edit    View    Help

┌──(kali㉿kali)-[~]
└─$ nano hdoc67.c
┌──(kali㉿kali)-[~]
└─$ gcc hdoc67.c
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 48
Enter second number: 18
GCD of 48 and 18 = 6
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 17
Enter second number: 13
GCD of 17 and 13 = 1
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 100
Enter second number: 25
GCD of 100 and 25 = 25
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 7
Enter second number: 7
GCD of 7 and 7 = 7
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 36
Enter second number: 60
GCD of 36 and 60 = 12
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 0
Invalid input. Enter positive integers only.
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter first number: 10
Enter second number: -4
Invalid input. Enter positive integers only.
┌──(kali㉿kali)-[~]
└─$
```